Programming Assignment 3

CSCI 4335/5360

Due: 16 Apr 2024 by 11:59 pm
April 2, 2026

1 Abstract

The objective of this task is to understand the reliable data transfer. You will use UDP to implement Go-Back-N protocol. Your implementation must be able to tolerate packet drops (assume there is no bit error or packet delay in the channel).

2 Introduction

In this PA, you will implement a transport protocol with properties equivalent to Go-Back-N to transfer text files from transmitter(Tx) to receiver (Rx). The protocol should be able to handle packet/ACK losses. For simplicity, we will consider the channel has no bit error or packet delay. The communication is one-directional, i.e, data packet will flow in only one direction from Tx to Rx, and the ACK for that data packet will be sent from Rx to Tx.

You have been provided with helper module namely, `pa3` with files packet.py and `cQueue.py`; and skeleton source files `tx.py` and `rx.py`. The `tx.py` declares a function

```
reliablyTransfer(rx_ip, rx_port, filename)
```

This function should read the `bytesToTransfer` bytes from `filename` to the receiver at `rx_ip: rx_port` correctly and efficiently, even if the network drops some of your packets. Similarly, it will receive ACK packets at the `tx_port`.
You also have `reliablyReceive(rx_ip, rx_port, destinationFile)`. This function is `reliablyTransfer`'s counterpart and should write what it receives to a file called `destinationFile`.

3 Packet Data Structure (packet.cpp)

A `Packet` class is provided to you. In reality, data in a packet needs to be serialized (converted to bytes) before sent through a socket. For sending text data (string type) across the network, this is easy, as you have done in PA1 and PA2. There are different ways to serialize an object in Python, as mentioned in the Object Serialization in Python webpage. We will use object serialization using python object as illustrated in the the `Packet` class. The `Packet` class looks similar to the following:

```
class Packet:
    def __init__(self, flag, seqnum, length, payload):
        self.flag = flag
        self.seqnum = seqnum
        self.length = length
```

```
        self.payload = payload

    def serialize(self):
        # more code here

    @classmethod
    def deserialize(cls, data):
        # more code here
```

The `flag` field indicates the following:

- flag = 1; the packet is a data packet (carrying payload)
- flag = 0; the packet is an ACK packet (no payload)
- flag = 2; the packet is a FIN packet or end of transmission packet (no payload)

For data packets, the `seqnum` denotes the seq number of the packet. All seq number ranges from 0 to 9. The sequence number of the first packet should be zero.

For ACK packets, the `seqnum` should carry the seq number for the last correctly in-order packet received. You will send an ACK for every packet received. For eg: the `seqnum` for first ACK packet will be zero.

`length` indicates the size of the payload. You may assume the maximum size of the payload to be 50. Please note that, for data packets, the last packet may have a length less than 50; depending on how many more bytes are left to send at the end of the file.

For ACK and FIN packets, the length should be set to 0.

`payload` is the bytes from the text file to transfer. For ACK and FIN packets, the payload should be empty.

4 Handling Window

Consider the maximum window size to be 5. To implement the window on the Tx side, you must use circular queue. The front of the queue will denote the oldest unacknowledged packet in the window, and rear denotes the latest unacknowledged packet in the window. Please note that, although the window index goes from 0 to 4, the seq number will go from 0 to 9.

Furthermore, to increase performance, you will send the packets in a pipeline (that is, send all the new packets in the window). Similarly, receive all the ACKs sent by the Rx and remove the acknowledged packets from the window and repeat these steps till all packets are sent and acks are received reliably.

5 Handling Timeout

There are multiple ways to implement timeout when a packet loss event occurs. For simplicity, you may chose to do the following:

- Set the timeout on the socket for `recvfrom` call using `socket.settimeout()` method. To know more, you may look into the code given in **Listing2-2** in Page 24 and 25 of the book Python Network Programming.
- Once timeout occurs, resend all packets starting with the packet for which the timed out occurred to the end of the window.

6 Tx Side: Go-Back-N Protocol

Upon execution, the tx program should be able to read atmost `bytesToTransfer` bytes (assume 50 bytes) of data from the specified `filename` and send it using Go-Back-N to the Rx. After all the contents of the file are sent successfully and corresponding ACKs have been received, the client should send a FIN packet to the Rx. Upon receiving all the ACKs for the sent data packets, and final ACK for the FIN packet, the Tx will close the connection.

To ensure reliable transmission, your program should implement the following:

1. If the Tx has a packet to send, it first checks to see if the window is full - that is, whether there are N outstanding unacknowledged packets. If the window is not full, the window is filled and the new packets in the window are sent. If the window is full, the client will wait for ACKs to be received for the sent packets and subsequently, the window is dequeued.
2. When Tx receives an ACK with sequence number n, the ACK will be taken as a cumulative acknowledgement, indicating that all packets with a sequence number upto and including n have been correctly received at the Rx. The packets with those sequence numbers will be dequeued from the window.
3. If timeout occurs, the Tx resends all packets that have been previously sent but that have not yet been acknowledged.

7 Rx side: Go-Back-N Protocol

Upon execution, the rx program should be able to write payload received (atmost 50 bytes of data) to the `destinationFile`. Your program should implement the following:

1. Check the sequence number of the packet
2. If the sequence number is the one that the Rx is expecting, then it should send an ACK packet back to the Tx with the sequence number of the received packet (indicating it has received all packets up to and including this packet).
3. In all other cases, it should discard the received packet and send ACK for the last correctly received in-order packet. Please note, in reality, the Rx sends an ACK for the last correctly received in-order packet.

4. After the Rx has received all data packets and a FIN packet from the Tx, it should send an ack for the FIN packet back to the Tx. Do not close the socket for the Rx as the Tx may have to resend FIN packet if the previous ACK is lost.

8 How to test your project?

8.1 Test 1:

You can use the basic test environment that you have setup in the previous PAs. However, the network performance will be really good with no packet loss, so you'll need to limit it. If your Tx side of the network interface inside the VM is enp0s3, then run (from inside the VM) the following command:

```
sudo tc qdisc del dev enp0s3 root 2>/dev/null
```

This will delete the existing tc rules.

```
sudo tc qdisc add dev enp0s3 root handle 1:0 netem delay 20ms loss 2%
```

followed by

```
sudo tc qdisc add dev enp0s3 parent 1:1 handle 10: tbf rate 20Mbit burst 10mb latency 1ms
```

will give you a 20Mbit, 20ms RTT link where every packet sent has 2% chance to get dropped. Simply replace the 2% with any number to apply your own artificial drops.

Note: You do not need to run these commands on the receiver side of VM, however, you may implement it if you want to drop ACK packets as well.

Once you run your program with the packet drop, check if the destinationFile on the Rx side is same as the file on the Tx side. You can use linux **diff** command.

8.2 Test 2:

Your Tx should generate two log files; namely TxSeqNum.log and TxAck.log.

Whenever a packet is sent, it's sequence number should be recorded in the `TxSeqNum.log`. This includes FIN packet sent by the Tx. You should insert "Timeout" when timeout occurs for a packet loss.

Whenever a packet is acknowledged, it's sequence number should be recorded in the `TxAck.log`. This includes the ACK for the FIN packet.

The format for these two log files is one number per line.

Similarly, your Rx should generate two log files; namely RxSeqNum.log and RxAck.log.

The log files should comply with the Go-Back-N protocol. I will grade your assignment based on the Test1 and Test2 correctness.

9 How to run your program?

The Tx side would do something like the following:

```
./tx.py -ip 192.168.56.102 -p 8001 -f data/large.txt
```

`data/large.txt` denotes the filepath of the data to be sent, `8001` is the Rx listening port, `192.168.56.102` is the ip address of the Rx.

The Rx side would do something like the following:

```
./rx.py -ip 192.168.56.102 -p 8001 -f data/receivedFile.txt
```

`dir2/receivedFile.txt` is the filepath of the destinationFile to be written, `8001` is the Rx listening port, `192.168.56.102` is the Rx ip address.

10 Tips

You may implement these steps and do `git commit` for each of the following steps. If your program is not fully correct, you will get partial grades based on each of the following steps.

1. First, write a program for a simple talker-listener program using UDP to send the file from Tx to Rx.
2. Modify your code such that the data you are sending from Tx to Rx is first made into a packet object and then serialized. Similarly, the Rx deserializes the received data to extract the payload and write it back to the destination file. Moreover, implement the end-of-transmission (FIN) packet as well at this step.
3. Modify your code to send an ACK from Rx for each pkt received. Moreover, modify the code to maintain window (circular queue) and update based on each ACK received.
4. Finally, implement Go-Back-N protocol with timeout event.
5. Add packet loss, and test your protocol (using section 8).

11 VM Setup: Using VirtualBox

For this course, we will use 64-bit Ubuntu 22.04.3 LTS virtual machine.

1. Install VirtualBox on your machine.
2. Download the Ubuntu 22.04.3 LTS from here:`https://ubuntu.com/download/desktop`
3. Follow the tutorial to install Ubuntu on VirtualBox: `https://ubuntu.com/tutorials/how-to-run-ubuntu-desktop-on-a-virtual-machine-using-virtualbox#1-overview`
4. After the Ubuntu installation process (within the VM), you should install the ssh server. You can do `sudo apt-get install openssh-server` once the OS is installed on the Ubuntu terminal. Use `apt-get` to install any programs you'll need like (`sudo apt-get install gcc make gdb valgrind iperf tcpdump net-tools wget`) which will be useful in future programming assignments.

12 Setup connection between VMs

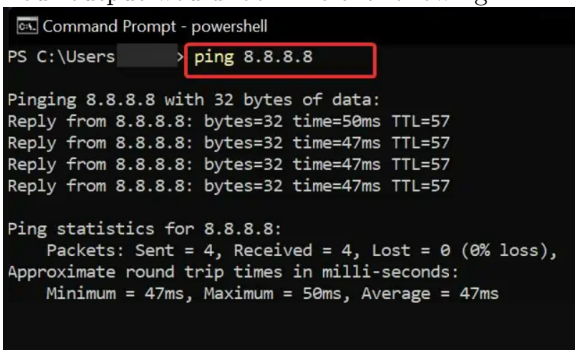
1. VirtualBox's default network setup is a NAT (which will learn later in the course) interface to the outside world, provided by the host computer. This allows the VM to access the Internet, but the host computer and other VMs will not be able to talk to each other. Therefore, you are going to replace NAT interface with one that allows those communications.

   However, before you make this change, make sure you install all the required softwares on your OS first as described in 3.4. After the change of the interface to "host-only", you will lose the internet. Optionally, there are ways to have two interfaces of the VM configured with NAT and host-only respectively, but I will leave it upto you to figure it out if you are interested.

2. Now it's time to replace the network interface. Make sure the VM is fully shut down, go to its Setting –> Network section. Switch the Adapter Type from NAT to host-only, and click ok. Restart, and the VM will now be able to talk to the other host-only VMs on the same computer, as well as the host computer (for ssh). You can test the connection using ping command. Run

   ```
   ping <ip address of the other VM/host machine>
   ```

   Your output would look like the following:

   Replace 8.8.8.8 with the ip address of the other VM or the host machine.

3. Finally, `sshfs` is a convenient way to mount your VM's filesystem within your host OS, as if it were a USB flash drive. This will help you to upload/download files between host and VM's easily. Follow the link here on how to set it up: `https://phoenixnap.com/kb/sshfs`

13 Deliverables

1. Create a README file with your group member names, the lines of code that each member has contributed for this assignment (one or more members may work on same section of the code), and instructions on how to run your code.

2. Upload your source files, log files and README as a single zip file on D2L.

3. Mention what steps work and what steps doesn't based on the Tips section in README file.

14 Evaluation

Automatic Deductions:	
Not Submitted	-100 pts
Code does not run/compile	-50 pts
Use of external reference (except textbooks, instructor notes) without citation	-100 pts
Unable to explain how your code works	-100 pts

Earn Points for the following:	
Code has a comment header with name, section, date	2 pts
Code organization, structure, and indention is appropriate (SHFT + ALT + F in VS Code)	1 pts
Code is well organized and meaningfully commented.	1 pts
Appropriate variable, method and file names used	1 pts
Followed the Deliverable instructions and filenames correctly	5 pts
Tx.py and Rx.py	
Send segments between Tx and Rx correctly (in pipeline), considering no packet loss	10 pts
Send segments between Tx and Rx correctly (in pipeline), considering packet loss	20 pts
Packet object is inserted into the window and sent correctly (using serialization and enqueue)	20 pts
ACKs are received correctly and packet objects are removed from window(using deserialization)	20 pts
Timeout is implemented correctly	10 pts
FIN packet feature is implemented correctly	10 pts